Blockchain Developer — Final Exam

Overview

This exam is split into two major pillars:

1. **Smart Contracts** — Build an ERC-4337 compliant smart account system with session key support, deployable and usable through a frontend.
2. **EVM Indexer** — Build a backend service that tracks ERC-4337 events emitted by the **EntryPoint v0.7** contract and displays them live in a frontend interface.

Both pillars share a single frontend. The smart account you build in Part 1 should be the one whose operations appear in the indexer feed of Part 2.

EntryPoint v0.7 address: `0x0000000071727De22E5E9d8BAf0edAc6f37da032`

Part 1 — Smart Contracts

1.1 — Smart Account Contract

Implement an ERC-4337 compliant **Smart Account** in Solidity. It must:

- Implement the `IAccount` interface, specifically `validateUserOp(UserOperation calldata userOp, bytes32 userOpHash, uint256 missingAccountFunds)` as defined by EntryPoint v0.7
- Support **at least two authentication methods**:
  - **ECDSA signature** — the account owner signs UserOperations with a standard EOA private key (the "default" login)
  - **Session keys** — a secondary address can be granted temporary execution rights, scoped to specific function selectors and/or a time-based expiry. Session keys must be revokable by the owner.
- Include an `execute(address target, uint256 value, bytes calldata data)` function that the EntryPoint calls after successful validation
- Emit a `SessionKeyAdded(address indexed key, uint256 expiry)` and `SessionKeyRevoked(address indexed key)` event for auditability

> The session key system is the core of this part. A session key is not an owner — it can only call what the owner explicitly allowed, for a limited time.

Deliverables:
- `SmartAccount.sol` — the account contract
- `SmartAccountFactory.sol` — a factory that deploys `SmartAccount` instances deterministically (using `CREATE2`) given an owner address and a salt. The factory must implement the `createAccount(address owner, uint256 salt)` and `getAddress(address owner, uint256 salt)` functions.
- Hardhat or Foundry project with compile scripts

1.2 — Demo Target Contract

Deploy a simple contract on a **testnet** (Sepolia recommended) that the smart account will interact with in the demo. At minimum, implement one of the following:

- A `Counter` contract with `increment()` and `getCount(address account)` — each smart account has its own counter
- An ERC-20 faucet contract where smart accounts can `claim()` tokens and `transfer()` them to other addresses

The contract must be verified on the testnet block explorer.

1.3 — Frontend Integration

Extend the frontend (the same one used for the indexer, or a dedicated page/tab) so that users can:

- **Deploy their smart account**: given a connected EOA wallet (e.g. via MetaMask), compute the counterfactual address using the factory's `getAddress`, and deploy it by sending a UserOperation with the `initCode` field set
- **Interact as owner**: sign and submit a UserOperation (e.g. call `increment()` on the Counter, or `transfer()` tokens) using their EOA as the signer — the operation must go through a **bundler** (you can use Pimlico, Alchemy's bundler, or a local Rundler instance)
- **Add a session key**: through the UI, the owner can register a new session key address with an expiry timestamp
- **Interact as session key**: a second connected wallet (or an in-browser generated keypair) uses the session key to submit a UserOperation without the owner's signature — the allowed action should be visibly scoped (e.g. only `increment()`, not `transfer()`)
- Display the smart account's current state (counter value or token balance) and refresh it after each operation

Deliverables:
- Frontend code with clear separation between owner flow and session key flow
- A short demo script or walkthrough in the README (step-by-step for the evaluator to reproduce the demo)

1.4 — Evaluation Criteria

Criteria
`validateUserOp` correctly validates both ECDSA and session key signatures
Session keys are properly scoped (selector and/or expiry) and revokable
Factory deploys accounts deterministically via CREATE2
Frontend: deploy account + submit UserOp as owner (end-to-end on testnet)
Frontend: session key registration + session key UserOp submission
Demo contract deployed and verified
Bonus: paymaster integration (sponsor gas for session key operations)

Part 2 — Technical Challenge: EVM Indexe

2.1 — The Indexer (Backend)

Build a backend service that:

- Connects to **Ethereum Mainnet** via a node provider (Alchemy, Infura, QuickNode, or a local node — your choice, justify it)
- Listens and indexes the following ERC-4337 events from the EntryPoint contract:
  - `UserOperationEvent(bytes32 indexed userOpHash, address indexed sender, address indexed paymaster, uint256 nonce, bool success, uint256 actualGasCost, uint256 actualGasUsed)`
  - `AccountDeployed(bytes32 indexed userOpHash, address indexed sender, address factory, address paymaster)`
  - `UserOperationRevertReason(bytes32 indexed userOpHash, address indexed sender, uint256 nonce, bytes revertReason)`
- Persists indexed data in a database (your choice, justify it)
- Exposes the indexed data through an API (REST or WebSocket — your choice, justify it)
- Handles **historical backfill** (from a configurable start block) and **real-time** new events
- Handles gracefully: RPC failures, reorgs, rate limiting

Deliverables:
- Source code with a clear README to run the project locally
- A `.env.example` file listing all required environment variables
- At minimum, handle the `UserOperationEvent`; the other two events are a bonus

---

2.2 — The Frontend

Build a frontend that:

- Displays indexed events in a **live feed** (auto-updates when new events arrive)
- For each `UserOperationEvent`, shows at minimum:
  - `userOpHash`
  - `sender` (the smart account)
  - `paymaster` (if any — highlight sponsored transactions)
  - `success` status
  - `actualGasCost` (in ETH, human-readable)
  - Block number and timestamp
- Includes a **basic stats panel**: total UserOps indexed, success rate, % sponsored by a paymaster
- Bonus: clickable row that opens the transaction on Etherscan

Deliverables:
- Frontend source code (any framework — justify your choice)
- Must be runnable locally alongside the backend

2.3 — Evaluation Criteria

| Criteria
|---|---|
| Indexer works end-to-end (events correctly captured and stored)
| Real-time frontend updates
| Code quality, structure, error handling
| README clarity (setup in under 10 minutes)
| Bonus: all 3 event types indexed + displayed
| Bonus: historical backfill with configurable start block

Part 3 — Architecture Defense

During the oral defense, you will be asked to explain and justify your architectural choices across **both pillars**. You must be able to answer questions such as:

On the indexer:
- Why did you choose this node provider (Alchemy / Infura / local node / etc.) ?
- Why did you use polling vs. WebSocket subscriptions for event listening ?
- Why did you choose this database (PostgreSQL / SQLite / Redis / etc.) — what are the trade-offs ?
- How does your system handle Ethereum **chain reorganizations (reorgs)** ?
- Why REST vs. WebSocket for your API ?
- How would you scale this indexer to handle multiple contracts or chains simultaneously ?

On the smart contracts:
- Walk me through exactly what happens when EntryPoint calls `validateUserOp` on your contract — line by line.
- How does your session key validation differ from the ECDSA owner validation at the bytecode level ?
- What prevents a session key from calling a function it was not granted access to? Where is that check enforced ?
- Why `CREATE2` for the factory — what does the counterfactual address property enable in the ERC-4337 flow ?
- What are the security risks of session keys, and how did you mitigate them ?
- How would you extend the session key system to support spending limits (e.g. max 0.1 ETH per session) ?

> ⚠ You are expected to have **real opinions** and **defend them**. "I used X because it was easy" is not a sufficient answer. Know your trade-offs.

Submission Instructions

- Submit a **single GitHub repository** (public or shared with the evaluator) containing:
  - `/contracts` — Solidity source, Hardhat or Foundry config, deployment scripts
  - `/indexer` — backend indexer code
  - `/frontend` — frontend code (shared between both pillars)

- `README.md` at the root with full setup instructions for all three components

> Any code copied from AI tools without understanding will be immediately apparent during the defense. You are allowed to use AI as a tool — but you must be able to explain every line.

Good luck. Build something you're proud of.